SimpleShell

Operating System : Assignment 2

Group Number:161

Group Member: Mihir Jaiswal

Implementation

Shell Loop

The shell repeatedly displays a prompt, reads a command from the user, and executes it. The loop continues until the user terminates the shell using Ctrl-C.

Main functions used:

• shell_loop() — It controls the main shell loop.
• read_user_input() — It reads the command which is entered by the user.
• launch() — It decides whether the command is a normal command or a pipeline.

Command Parsing

The input command is divided into individual arguments using whitespace as the delimiter.
The parse_command() function uses strtok() to separate the command and its arguments.

Example:

```
ls -l

→ ls
 -l
```

Creation of the process and execution

For a normal command, the shell creates a child process using fork(). The child process executes the requested command using execvp(). The parent process waits for the child using waitpid(). The implementation stores the child PID and exit status for the history information.

```
Shell
|
| fork()
|
+---- Child ----> execvp()
|
+---- Parent ---> waitpid()
```

Implementation of the pipeline

The shell supports commands connected using the pipe operator |. The implementation creates pipes using pipe(). dup2() is then used to connect the standard output of one process to the standard input of the next process. For a pipeline with multiple commands, a separate child process is created for each command.

```
cat helloworld.c | grep print | wc -l

cat helloworld.c
|
```

```
v
grep print
|
v
wc -l
```

Command History

The shell maintains a history of commands entered by the user. The history stores the command, process PID(s), execution time, duration, and exit status. The history command displays the commands previously entered in the shell.

Execution Time

The execution duration of commands is measured using clock_gettime() with CLOCK_MONOTONIC. The shell records the start time before execution and the end time after execution. The difference between these timestamps gives the execution duration. The execution timestamp is also stored using time() and displayed in a human-readable format.

Signal Handling

The shell handles Ctrl-C using the SIGINT signal. The parent shell registers a signal handler and changes its running state when SIGINT is received. Child processes restore the default SIGINT behaviour so that they can be interrupted normally.

```
signal(SIGINT, process_interrupt);
```

Error Handling

The implementation checks errors for important system calls and operations, including fork(), pipe(), dup2(), execvp(), waitpid(), clock_gettime(), and signal registration. When an error occurs, an appropriate error message is displayed using perror() or fprintf(). Invalid or empty commands in a pipeline are also detected.

Limitations

The implementation supports the functionality required by the assignment but does not implement a complete Unix shell. The following features are not supported:

- Input redirection <
- Output redirection >
- Append redirection >>
- Background execution using &
- Quoted arguments
- Escaped characters
- Shell variables
- Command substitution
- Shell scripting
- Built-in commands such as cd

Testing Part

The shell was tested using normal commands, commands with arguments, executables, and pipelines.

```
ls
```

```
ls /home
ls -l
echo hello
wc -l fib.c
wc -c fib.c
grep printf helloworld.c
 ./fib 40
 ./helloworld
sort fib.c
uniq file.txt
```

Pipeline tests:

```
cat fib.c | wc -l
cat helloworld.c | grep print | wc -l
```

The history command and Ctrl-C termination were also tested

Conclusion

The SimpleShell implements the required basic functionality of a Unix shell, including command execution, process creation, pipelines, command history, execution timing, PID tracking, and signal handling. The implementation was tested with the commands and pipeline examples specified in the assignment.

GitHub Repository

Private GitHub Repository: https://github.com/Mihir2024347/SimpleShell